MeshMapper: Installation & Setup Guide

This firmware and installation guide is optimized for Raspbian Bullseye with Desktop environment. It can be run headless, but the desktop version includes native libraries required for image processing.

---

1. Hardware & System Configuration (Raspi-Config)

You need to optimize the system memory allocation and localization before installing dependencies.

sudo raspi-config

* Navigate to Performance Options --> GPU --> Change value to 256. (This increases GPU memory allocation, which is critical for processing captured high-resolution photos).
* Navigate to Localization Settings --> WLAN Country --> Select your local region. (Ensures correct Wi-Fi frequency compliance).
* Save changes and reboot the Raspberry Pi.

---

2. MeshMapper Directory Architecture

Create the foundational folder structure required for the MeshMapper local server, temporary caches, and update modules.

```
mkdir -p /home/pi/MeshMapper/scans \
        /home/pi/MeshMapper/files \
        /home/pi/MeshMapper/settings \
        /home/pi/MeshMapper/tmp \
        /home/pi/MeshMapper/updates
```

---

3. WLAN Interface Network Configuration

To ensure persistent network interface rules, append the configuration to the network interfaces file.

sudo nano /etc/network/interfaces

Add the following block to the end of the file:

auto wlan0
iface wlan0 inet manual
wpa-conf /etc/wpa_supplicant/wpa_supplicant.conf

---

4. Samba Shared Network Drive Setup (Highly Recommended)

Setting up a Samba share creates a shared network drive, allowing you to drop custom datasets directly into the MeshMapper ecosystem via your local network.

# Install Samba packages
sudo apt-get update && sudo apt-get install -y samba samba-common-bin

# Configure Samba global settings
sudo nano /etc/samba/smb.conf

Inside the file, verify or modify these permission parameters:

read only = no
create mask = 0775
directory mask = 0775

(If you are connecting via a Windows machine, ensure wins support = yes is uncommented or added).

Append the explicit MeshMapper share definitions to the absolute bottom of the file:

[PiShare]
comment=Raspberry Pi MeshMapper Share
path=/home/pi/
browseable=Yes
writeable=Yes
only guest=no
create mask=0777
directory mask=0777
public=yes

Set the network access password for user pi and open global file system permissions:

sudo smbpasswd -a pi
sudo chmod -R 777 /home/pi/MeshMapper

---

5. DSLR Camera Control Interface (GPhoto2)

Required if you plan to trigger external DSLR cameras via a USB interface instead of standard Pi camera modules.

sudo apt install -y libgphoto2-dev gphoto2
sudo pip3 install -v gphoto2

---

6. Node-RED Core Installation & Orchestration

Node-RED serves as the visual logic controller for the MeshMapper interface.

Download and execute the Node-RED installation script
bash <(curl -sL https://raw.githubusercontent.com/node-red/linux-installers/master/deb/update-nodejs-and-nodered)
Confirm the prompts with 'y'

Systemd Service Modification
To ensure Node-RED handles low-level hardware communication correctly, it must be timed with the backend Flask service.

sudo nano /lib/systemd/system/nodered.service

Modify the [Unit] and [Service] sections precisely as follows:

[Unit]
Wants=network.target flask.service
After=flask.service

[Service]
User=root
#Group=pi

Initialize Node-RED Settings
Initialize the setup profile:
sudo node-red admin init

Recommended initialization selections:
* Security: No (Adjust to Yes if deployed on public networks)
* Project: No
* Flows File Settings: Press [Enter] (Default)
* Passphrase: Press [Enter] (None)
* Theme: default
* Text editor: default
* External Modules: Yes

MeshMapper Path Overrides
Open the main configuration file to map Node-RED directly into the MeshMapper ecosystem:

sudo nano /root/.node-red/settings.js

Uncomment and update the following target variables:

```
userDir: '/home/pi/MeshMapper/settings/.node-red/',
uiPort: process.env.PORT || 80,
httpAdminRoot: '/editor',
httpStatic: '/home/pi/MeshMapper/',
ui: { path: "" },
functionGlobalContext: {
    os:require('os'),
    path:require('path'),
    fs:require('fs')
},
```

Enable the automatic system service and reboot:
```
sudo systemctl enable nodered.service
sudo reboot
```

Install Supplementary Nodes
Navigate to your redefined configurations folder to install required node dashboards and processing elements:

```
cd /home/pi/MeshMapper/settings/.node-red/
sudo npm i node-red-dashboard node-red-contrib-python3-function node-red-node-ui-table
```

---

7. Libcamera Vendor Calibration (Arducam IMX519 Integration)

Skip this step if you are utilizing native Raspberry Pi standard camera modules.

Fetch and prepare driver script
wget -O install_pivariety_pkgs.sh https://github.com/ArduCAM/Arducam-Pivariety-V4L2-Driver/releases/download/install_script/install_pivariety_pkgs.sh
chmod +x install_pivariety_pkgs.sh

# Run driver setup dependencies
sudo apt update
./install_pivariety_pkgs.sh -p libcamera_dev
./install_pivariety_pkgs.sh -p libcamera_apps
./install_pivariety_pkgs.sh -p imx519_kernel_driver

---

8. MeshMapper Firmware Deployment

> CRITICAL NOTE: The original URLs target raw files specifically compiled for OpenScan hardware and user flows. You must manually replace these repository URLs with your own hosted repository targets containing your custom-coded variations (MeshMapper.py, mesh_fla.py, etc.). If you pull down the OpenScan source files directly, they will execute internally under "OpenScan" paths and completely break your new directory structure.

# Browser UI Flow Configuration
sudo wget https://raw.githubusercontent.com/YourRepo/MeshMapper/main/flows.json -O /home/pi/MeshMapper/settings/.node-red/flows.json

# Core Python Firmware Modules
sudo wget https://raw.githubusercontent.com/YourRepo/MeshMapper/main/MeshMapper.py -O /usr/lib/python3/dist-packages/MeshMapper.py

# Local Flask Server Routing Script
sudo wget https://raw.githubusercontent.com/YourRepo/MeshMapper/main/fla.py -O /home/pi/MeshMapper/files/fla.py

# Specialized Boot System Configuration
sudo wget https://raw.githubusercontent.com/YourRepo/MeshMapper/main/config.txt -O /boot/config.txt

# Lens Autofocus Drivers (Sensor Specific)
sudo wget https://raw.githubusercontent.com/YourRepo/MeshMapper/main/Arducam.py -O /usr/lib/python3/dist-packages/Arducam.py

# User Interface Branding Assets
sudo wget https://raw.githubusercontent.com/YourRepo/MeshMapper/main/logo.jpg -O /home/pi/MeshMapper/files/logo.jpg

---

9. Flask Local Micro-Server Initialization

Create a background daemon service script so your Python backend processes start up smoothly during boot.

sudo nano /lib/systemd/system/flask.service

Paste the following configurations:

[Unit]
Description=MeshMapper Photo Service
After=multi-user×target

[Service]
#ExecStartPre=/bin/sleep 5
ExecStart=/usr/bin/python3 /home/pi/MeshMapper/files/fla.py
StandardOutput=inherit
StandardError=inherit
Restart=always
RestartSec=5
User=root

[Install]
WantedBy=multi-user×target

Register, start, and confirm the service daemon:
sudo systemctl daemon-reload
sudo systemctl enable flask.service
sudo systemctl start flask.service

---

10. System Explanations & Hardware Tweaks

sudo nano /boot/config.txt
Line Entry: hdmi_blanking=2

> WHY IS THIS HERE?
> When running a Raspberry Pi setup intensively for 3D scanning or photography capture automation, the system resources are stretched thin. If the system drops output signaling to an unmonitored HDMI port, it triggers internal software interrupts and desktop window manager polling loops to look for a monitor. This can cause severe latency spike anomalies, micro-stuttering during camera captures, or unexpected drops in frame rate.
>
> Setting hdmi_blanking=2 completely disables the video output hardware interface when a screen is absent, preventing system resource interrupts from interfering with your time-sensitive photography cycles.